

Tarea 5

Nombre: Juan Pablo Granado Duval

Matrícula: 2026-0246

Materia: Programación para Mecatrónicos

Maestro: Carlos Pichardo

Fecha: 4/ 7/ 2026

Carrera: Mecatrónica

Codificación de Caracteres y Formatos de Archivo

Codificación de Caracteres

1. ASCII (American Standard Code for Information Interchange)

Origen: Estandarizado en 1963, revisado hasta 1986.

Características:

- Usa 7 bits por carácter, 128 valores posibles, 0–127.
- Se almacena típicamente en 1 byte (8 bits), dejando el bit más significativo en 0.
- Incluye caracteres de control: salto de línea (\n), tabulación (\t), retorno de carro (\r), etc.
- Incluye caracteres imprimibles (32–126): letras mayúsculas/minúsculas (A-Z, a-z), dígitos (0-9), signos de puntuación y símbolos básicos.
- Solo cubre el inglés básico. No soporta tildes, eñes, ni alfabetos no latinos, árabe, chino, cirílico, etc.

2. UTF-8 (Unicode Transformation Format - 8 bits)

Origen: Creado por Ken Thompson y Rob Pike en 1992-1993, hoy es la codificación dominante en la web.

Características:

- Codificación de longitud variable: usa entre 1 y 4 bytes por carácter.
- Compatible con ASCII: los primeros 128 caracteres Unicode se codifican exactamente igual que en ASCII (1 byte).

Estructura de bytes:

Rango Unicode	Bytes usados	Patrón de bits
U+0000 – U+007F	1 byte	0xxxxxxx
U+0080 – U+07FF	2 bytes	110xxxxx 10xxxxxx
U+0800 – U+FFFF	3 bytes	1110xxxx 10xxxxxx 10xxxxxx
U+10000 – U+10FFFF	4 bytes	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Ventajas:

- Ahorra espacio para textos en inglés/latino, mayoría de caracteres en 1 byte.
- No requiere “byte order mark” o “BOM” obligatorio.
- Es el estándar de facto en internet.

3. UTF-16 (Unicode Transformation Format - 16 bits)

Origen: Sucesor de UCS-2, usado por Windows, Java y JavaScript internamente.

Características:

- Codificación de longitud variable: usa 2 o 4 bytes por carácter.
- Los caracteres del Plano Básico Multilingüe (BMP) (la mayoría de idiomas comunes) usan 2 bytes.
- Los caracteres fuera del BMP (emojis, alfabetos históricos) usan 4 bytes mediante pares subrogados (surrogate pairs).
- Tiene variantes Big Endian (UTF-16BE) y Little Endian (UTF-16LE), indicadas con un BOM al inicio del archivo.

Ventaja: Más eficiente que UTF-8 para textos en idiomas asiáticos, ya que la mayoría de sus caracteres caben en 2 bytes.

Desventaja: Menos compatible con sistemas antiguos basados en ASCII/byte único.

Comparación rápida:

Codificación	Bits/carácter	Compatible ASCII	Uso típico
ASCII	7 (1 byte)	—	Sistemas legados, texto en inglés
UTF-8	8–32 (1-4 bytes)	Sí	Web, APIs, Linux, JSON
UTF-16	16–32 (2-4 bytes)	No	Windows, Java, JavaScript (interno)

Formatos de Archivo

1. JSON (JavaScript Object Notation)

Origen: Popularizado a inicios de los 2000s por Douglas Crockford, estandarizado como ECMA-404.

Características:

- Formato de texto ligero para intercambio de datos.
- Basado en pares clave-valor y estructuras anidadas.
- Tipos de datos soportados: objetos {}, arreglos [], cadenas, números, booleanos (true/false) y null.

Ejemplo:

```
{
  "nombre": "Ana",
  "edad": 28,
  "activa": true,
  "hobbies": ["leer", "correr"]
}
```

- Usos: APIs REST, archivos de configuración, almacenamiento de datos ligero.
- Ventajas: legible por humanos, fácil de parsear en casi todos los lenguajes de programación.
- Codificación: por defecto se recomienda UTF-8.

2. XML (eXtensible Markup Language)

Origen: Estandarizado por el W3C en 1998.

Características:

- Basado en etiquetas (tags) similares a HTML, pero personalizables.
- Estructura jerárquica con nodo raíz, elementos y atributos.

Ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<persona>
  <nombre>Ana</nombre>
  <edad>28</edad>
  <activa>true</activa>
</persona>
```

- Usos: documentos de configuración, documentos empresariales, formatos como .docx/.xlsx.
- Ventajas: muy estructurado, soporta esquemas de validación, namespaces.
- Desventajas: más verboso que JSON, mayor tamaño de archivo, parseo más pesado.

3. CSV (Comma-Separated Values)

Origen: Formato usado desde los años 70, sin estandarización formal hasta el RFC 4180 (2005).

Características:

- Formato de texto plano tabular.
- Cada línea representa una fila; los valores se separan por comas o a veces punto y coma, tabulaciones, etc.

Ejemplo:

```
nombre,edad,activa
Ana,28,true
Luis,35,false
```

- Usos: exportación/importación de datos en hojas de cálculo (Excel, Google Sheets), bases de datos, análisis de datos.
- Ventajas: extremadamente simple, ligero, compatible con casi cualquier software.
- Desventajas: no soporta estructuras jerárquicas o anidadas; ambigüedades con comas dentro de valores (se resuelve con comillas); sin estándar único para separadores o codificación.

Comparación rápida:

Formato	Estructura	Legibilidad	Tamaño	Uso típico
JSON	Jerárquica (objetos/arreglos)	Alta	Compacto	APIs, configuración
XML	Jerárquica (etiquetas)	Media	Voluminoso	Documentos empresariales
CSV	Tabular (filas/columnas)	Alta	Muy compacto	Hojas de cálculo, datasets